Submodule Reuse Map

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Catalogue-first reuse map binding each Helix OTA component to the verified submodule(s) that satisfy it (reuse / extend / new), with the decoupled boundary and any upstream additions each binding requires. Specifies the six NEW submodules to be created. Derived from the master design (`2026-06-07-helix-ota-design.md` §10) and the canonical catalogue (`documentation_standards.md` §9).
Issues	Exact HelixConstitution clause numbering is UNVERIFIED (carried from the corpus convention). Some catalogue submodules expose capabilities Helix OTA needs but whose precise public surface has not been inspected in this revision; those bindings are marked UNVERIFIED where the satisfying API has not been confirmed. The six NEW repos are not yet created.
Fixed	N/A (initial revision).
Continuation	Inspect each reuse/extend submodule's actual public API to confirm it satisfies the stated boundary and remove the UNVERIFIED tags; finalize the NEW-submodule list immediately before repo creation (master §10) and create PUBLIC repos on GitHub + GitLab per locked decision D4; fold any approved upstream additions back into the owning submodule rather than forking.

Table of contents

> The table-of-contents requirement is mandated by HelixConstitution §11.4.61 (UNVERIFIED clause number). This document carries its ToC immediately after the metadata table.

---

1. Purpose and scope

This document maps every Helix OTA component to the catalogue submodule(s) that satisfy it, classifies each binding as **reuse**, **extend**, or **new**, states the decoupled boundary each binding must respect (§11.4.28, UNVERIFIED), and records any **upstream additions** a binding would require in the owning submodule.

It is normative for component-to-submodule allocation. It honors **catalogue-first** (§11.4.74, UNVERIFIED): a component MUST be satisfied by an existing catalogue submodule where one exists; a NEW submodule is justified only when no catalogue submodule covers the need. Only the verified canonical submodule names from `documentation_standards.md §9` are used; no submodule name is invented (§7.1 / §11.4.6 / §11.4.123, UNVERIFIED clause numbers).

The twelve components covered are those named in the operator brief and the master design (§4–§5): auth, artifact intake/validation, storage, rollout engine, device registry, telemetry, transport, notifications, config, caching, docs export, and the Android client.

2. Legend and conventions

- **reuse** — consume the catalogue submodule as-is; no upstream change required.
- **extend** — consume the catalogue submodule, but the binding needs an *upstream addition* (new interface, adapter, or option) contributed back to that submodule (not forked).
- **new** — no catalogue submodule covers the need; satisfied by a NEW submodule (§4).
- **Boundary** — the decoupling contract the binding must respect (one purpose, well-defined interface, independently testable; §11.4.28, UNVERIFIED).
- **Upstream additions** — the concrete change a binding requires in the owning submodule, or *None* when pure reuse.
- `UNVERIFIED` — tags any claim, capability, or clause number not confirmed from a real source in this revision (§7.1 / §11.4.6 / §11.4.123, UNVERIFIED clause numbers). In particular, the precise public API of each catalogue submodule has not been inspected in this revision; “satisfies” claims about specific submodule capabilities are UNVERIFIED unless otherwise confirmed.

Canonical catalogue names used below (verified set, from documentation_standards.md §9): auth, security, database, Storage, observability, eventbus, ratelimiter, middleware, http3, mdns, recovery, Herald, config, discovery, cache, docs_chain / Document / Formatters, containers; KMP: Auth-KMP, Security-KMP, Storage-KMP, Config-KMP.

3. Component → submodule reuse map

Component	Catalogue submodule(s)	NEW submodule(s)	Class	Boundary	Upstream additions
Auth (OAuth2/ JWT, RBAC, API keys, audit)	auth, security, middleware; KMP: Auth-KMP, Security-KMP	—	reuse	Identity, token issue/verify, RBAC, and request-auth middleware stay in auth/ security/ middleware; Helix OTA only wires policies and routes — no auth logic re-implemented. (UNVERIFIED: that auth already exposes OAuth2/JWT + RBAC as Helix OTA needs.)	None expected; if device-token-bound-to-hardware-id (master §6) is not present, add a device-identity token option upstream in security/ Security-KMP (extend).
Artifact intake / validation	Storage (blob staging), security (hash/ signature primitives), middleware (upload)	ota-artifact-validator, ota-protocol (manifest schema)	new + reuse	Validation pipeline (structure → hash → signature → version monotonicity → target compatibility, master §5) lives in ota-artifact-validator; it depends on security for crypto primitives and ota-protocol for the manifest schema. No transport in the validator.	None in catalogue; new logic is contained in ota-artifact-validator. (UNVERIFIED: security exposes the SHA-256/512 + signature-verify primitives the validator needs.)

Component	Catalogue submodule(s)	NEW submodule(s)	Class	Boundary	Upstream additions
Storage (artifact blobs, MinIO/S3)	`Storage`; KMP: `Storage-KMP`	—	reuse	Object/blob persistence behind a storage port; Helix OTA holds no direct S3/MinIO SDK calls outside the `Storage` abstraction. (UNVERIFIED: `Storage` provides an S3/MinIO-compatible backend.)	None expected; if a streaming/range-get for large OTA `.zip` is missing, add it upstream in `Storage` (extend).
Rollout engine (staged %, halt/advance)	`database` (phase/cohort state), `observability`/`Herald` (signals consumed)	`ota-rollout-engine`	new	Staged-rollout + halt/advance logic is OS-agnostic and HTTP-free; engine state persists via a storage port backed by `database`; it consumes telemetry signals but contains no ingest. (master §8)	None in catalogue; rollout logic is new in `ota-rollout-engine`.
Device registry / inventory	`database`, `discovery`, `mdns`	`ota-protocol` (device + status types)	reuse	Device records, groups, and inventory persist via `database`; on-network device presence/lookup via `discovery`/`mdns`; wire types come from `ota-protocol`. Registry exposes a typed device port; no rollout or telemetry logic.	None expected. (UNVERIFIED: `discovery`/`mdns` fit the device-presence use case for the Android-on-Orange-Pi target.)
Telemetry (event ingest, health, metrics)	`observability`, `Herald` (alerting)	`ota-telemetry-schema`	new + reuse	Event/metric **schema + codecs** live in `ota-telemetry-`	None in catalogue; schema is new. Alert routing wired in `Herald`.

Component	Catalogue submodule(s)	NEW submodule(s)	Class	Boundary	Upstream additions
				`schema` (shared server + agent); pipeline/export and alerting reuse `observability` + `Herald`. Schema module carries no transport or storage. (master §9)	
Transport (HTTP/3→HTTP/2, Brotli, REST)	`http3`, `middleware`, `ratelimiter`, `recovery`	`ota-protocol` (REST/wire contracts)	reuse	`http3` provides the drop-in `net/http.Handler` with HTTP/2 fallback (master §3); `middleware` carries Brotli/gzip negotiation, `ratelimiter` throttling, `recovery` panic-recovery. Transport carries no business logic; request/response contracts come from `ota-protocol`.	None expected; if Brotli content-negotiation is not already in `middleware`, add it upstream (extend). (UNVERIFIED: Brotli negotiation present in `middleware`.)
Notifications (alerts on failure/halt)	`Herald`, `eventbus`	—	reuse	Alert/notification dispatch via `Herald`; internal fan-out of domain events via `eventbus`. Helix OTA publishes events; it does not implement a notifier. (UNVERIFIED: `Herald` covers the required alert channels.)	None expected.
Config (poll interval + jitter,	`config`; KMP: `Config-KMP`	—	reuse	All runtime configuration (e.g. 15 min +	None expected.

Component	Catalogue submodule(s)	NEW submodule(s)	Class	Boundary	Upstream additions
runtime config)				jitter poll, master §5/D7) flows through `config` (server) and `Config-KMP` (agent). No bespoke config parsing in Helix OTA modules.	
Caching (optional Redis)	`cache`	—	reuse	Optional caching layer behind the `cache` abstraction, used only where a measured need exists (master §3). No direct Redis client outside `cache`.	None expected.
Docs export (PDF/ HTML/ DOCX, Mermaid renders)	`docs_chain` / `Document` / `Formatters`, `containers` (substrate)	—	reuse	Multi-format export pipeline (master §12) reuses `docs_chain`/ `Document`/ `Formatters`; runs on the `containers` substrate (§11.4.76, UNVERIFIED). Source of truth stays Markdown + Mermaid; renders are derived.	None expected; if a Mermaid→draw.io/ UML render target is missing, add it upstream in the export chain (extend). (UNVERIFIED: render targets available.)
Android client (poll/ download/ verify/apply/ report)	KMP: `Auth-KMP`, `Security-KMP`, `Storage-KMP`, `Config-KMP`	`ota-android-agent`, `ota-update-engine-bridge`, `ota-protocol`, `ota-telemetry-schema`	new + reuse	`ota-android-agent` (KMP) orchestrates poll/ download/verify/ apply/report, consuming `ota-protocol` (wire types), `ota-update-engine-bridge` (apply via AOSP `update_engine`/ `boot_control`), `ota-telemetry-`	None in catalogue; device-apply + agent logic are new.

Component	Catalogue submodule(s)	NEW submodule(s)	Class	Boundary	Upstream additions
				`schema` (reports), and the KMP catalogue submodules for auth/crypto/ storage/config. Agent holds no server logic; bridge is Android-only and thin.	

4. NEW submodules (decoupled boundaries)

These six NEW submodules are justified because no catalogue submodule (§3) covers their purpose. Per locked decision D4 (master §2) they get **PUBLIC** repos created on GitHub + GitLab, with the final list confirmed in the MVP spec immediately before creation (master §10). Boundaries below mirror master §10 and the decoupling principle (§11.4.28, UNVERIFIED).

NEW submodule	Purpose	Decoupled boundary
`ota-protocol`	Shared wire types, manifest schema, status/ event enums (Go + KMP).	Pure contracts only — no business logic, no transport, no storage. Consumed by transport, artifact intake, device registry, and the Android client.
`ota-artifact-validator`	Structure / hash / signature / metadata validation pipeline for OTA artifacts.	OS-aware via plugins; **no transport**. Depends on `security` (crypto primitives) and `ota-protocol` (manifest schema). Independently testable on raw artifact bytes.
`ota-rollout-engine`	Staged-rollout + halt/ advance logic, OS-agnostic.	**No HTTP** — pure engine plus a storage port (backed by `database`). Consumes telemetry signals; emits phase decisions. Independently testable with a fake clock + fake telemetry.
`ota-update-engine-bridge`	Wrapper over AOSP `update_engine` / `boot_control` for	**Android-only**, thin and testable. No polling, no networking, no business policy — only the apply/

NEW submodule	Purpose	Decoupled boundary
	applying updates to the inactive A/B slot.	verify bridge surface. Consumed by `ota-android-agent`.
`ota-android-agent`	KMP device agent: poll, download, verify, apply, report.	Consumes `ota-protocol` + `ota-update-engine-bridge` + `ota-telemetry-schema` and the KMP catalogue submodules. No server-side logic; the only OS-apply path is the bridge.
`ota-telemetry-schema`	Telemetry event / metric schema + codecs.	Shared by server + agents; **no transport, no storage**. Pure schema + codecs. Reused by the telemetry pipeline (`observability`) and the Android agent.

5. Upstream additions summary

Most bindings are pure **reuse** (no upstream change). The bindings that *may* require an upstream addition (classified **extend**, contributed back — never forked) are, each UNVERIFIED pending inspection of the owning submodule's actual surface:

- `security` / `Security-KMP` — device-identity token bound to hardware id (Android KeyStore), if not already present (Auth component). (UNVERIFIED)
- `Storage` — streaming/range-get for large OTA `.zip` artifacts, if not already present (Storage component). (UNVERIFIED)
- `middleware` — Brotli content-negotiation with gzip fallback, if not already present (Transport component). (UNVERIFIED)
- `docs_chain` / `Document` / `Formatters` — Mermaid→draw.io/UML render target, if not already present (Docs export component). (UNVERIFIED)

No upstream addition is presented as confirmed-needed; each is conditional on inspecting the submodule first (see Continuation). New behavior with no catalogue home is contained entirely in the six NEW submodules (§4), keeping catalogue submodules unforked.

6. Catalogue-first compliance

Rule (UNVERIFIED clause numbers)	How this map honors it
§11.4.74 catalogue-first	Every component is satisfied by a catalogue submodule where one exists (§3); a NEW submodule is used only where the catalogue has no cover, and

Rule (UNVERIFIED clause numbers)	How this map honors it
	that absence is the stated justification (§4).
§11.4.28 decoupling	Each binding states a single-purpose, well-defined, independently-testable boundary (§3, §4).
§11.4.76 containers substrate	Docs export and dev/build run on the `containers` substrate (§3 Docs export). (UNVERIFIED)
D4 PUBLIC new repos	The six NEW submodules get PUBLIC GitHub + GitLab repos, listed before bulk creation (§4).
§7.1 / §11.4.6 / §11.4.123 anti-bluff	No invented submodule names; unconfirmed capabilities/claims tagged UNVERIFIED (§7).

7. Anti-bluff notes

Per HelixConstitution §7.1, §11.4.6, and §11.4.123 (UNVERIFIED clause numbers):

- Submodule names are taken **only** from the verified catalogue (`documentation_standards.md` §9); none is invented. The dashboard-React submodules mentioned in master §10 (`UI-Components-React`, `Dashboard-Analytics-React`, `Auth-Context-React`) are **not** on the verified catalogue §9 and are therefore deliberately **omitted** from this map rather than asserted as catalogue submodules (UNVERIFIED whether they belong to the catalogue).
- The six NEW submodule names and boundaries are reproduced from the master design (§10), not invented here; they remain pending repo creation.
- The precise public API of each catalogue submodule has **not** been inspected in this revision; every “satisfies” / “exposes” claim about a specific submodule capability is tagged UNVERIFIED, and confirming them is tracked in the Continuation row.
- HelixConstitution clause numbers (§11.4.74, §11.4.28, §11.4.76, §11.4.61, §7.1, §11.4.6, §11.4.123) are carried from the corpus convention and are UNVERIFIED against the authoritative constitution text.